

APLICACIÓN DEL PATRÓN ABSTRACT FACTORY CALCULADORA

Angie Nathaly Cisneros Guerrero 20242020084

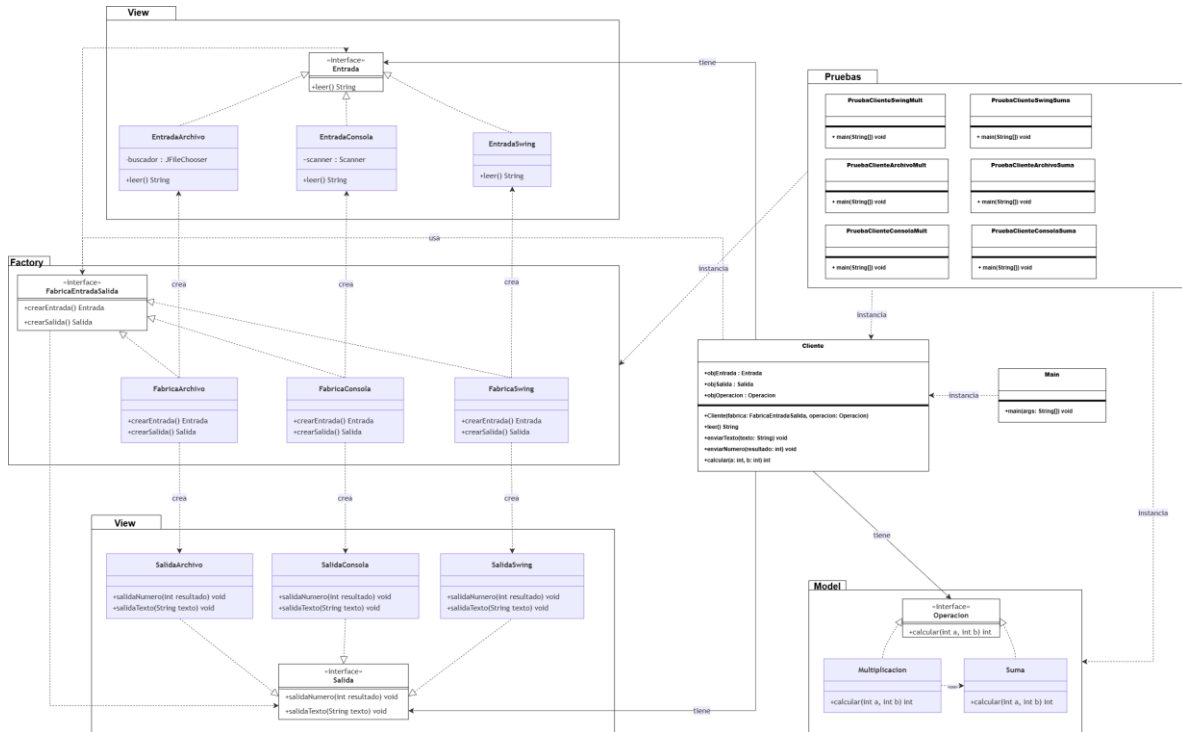
Universidad Distrital Francisco José de Caldas
Ingeniería de Sistemas
Modelos de Programación Grupo 020-84
Bogotá D.C
Agosto de 2026

Índice

1.	Diagrama UML.....	3
2.	Explicación de las Clases:.....	4
	2.1. Implementación de las Fábricas :	4
	2.1.1. Fabrica Archivo:	4
	2.1.2. Fabrica Consola:.....	4
	2.1.3. Fabrica Swing:.....	5
	2.2. Implementación de los Productos:.....	5
	2.2.1. Producto Concreto Consola:	6
	2.2.2. Producto Concreto Swing:	7
	2.2.3. Producto Concreto Archivo:.....	7
	2.3. Implementación de las Operaciones:	9
	2.3.1. Suma :	9
	2.3.2. Multiplicación:	9
3.	Código Cliente Explicación:.....	10
4.	Pruebas para cada producto y operación:.....	12
	4.1. Consola:	12
	4.2. Swing:.....	13
	4.3. Archivo:.....	14
5.	Código:	16
	5.1. Link del repositorio:	16
	5.2. Estructura de Paquetes:	16

1. Diagrama UML

La aplicación implementa el patrón de diseño creacional Abstract Factory con el objetivo principal de desacoplar la lógica matemática de las interfaces de usuario (entradas y salidas). Esto permite que la calculadora funcione en múltiples entornos sin que el código central deba ser modificado.



2. Explicación de las Clases:

2.1. Implementación de las Fábricas :

Primero se creó la fábrica abstracta **FabricaEntradaSalida** que declara la creación de los productos abstractos entrada y salida:

```
package calculadorasumas.fa.Factory;

import calculadorasumas.fa.View.Entrada;
import calculadorasumas.fa.View.Salida;

/**
 *
 * @author Nath
 */
public interface FabricaEntradaSalida {

    public Entrada crearEntrada();
    public Salida crearSalida();
}
```

Y a partir de esa interfaz se crean las fábricas concretas que implementan la fábrica abstracta para devolver productos concretos.

2.1.1. Fabrica Archivo:

Esta clase implementa la interfaz FabricaEntradaSalida y es la responsable de crear la familia de productos compatibles diseñados para el manejo de la entrada por archivos.

Retorna los Productos concretos Entrada Archivo y Salida Archivo

```
5 package calculadorasumas.fa.Factory;
6
7 import calculadorasumas.fa.View.Entrada;
8 import calculadorasumas.fa.View.EntradaArchivo;
9 import calculadorasumas.fa.View.Salida;
10 import calculadorasumas.fa.View.SalidaArchivo;
11
12 /**
13  *
14  * @author
15  */
16 public class FabricaArchivo implements FabricaEntradaSalida{
17
18     @Override
19     public Entrada crearEntrada() {
20         return (Entrada) new EntradaArchivo();
21     }
22
23     @Override
24     public Salida crearSalida() {
25         return (Salida) new SalidaArchivo();
26     }
27
28 }
```

2.1.2. Fabrica Consola:

Clase que implementa la interfaz FabricaEntradaSalida creandola familia de productos para recibir datos e interactuar a través de la consola del sistema.

Retorna los Productos concretos Entrada Consola y Salida Consola

```

5 package calculadorasumas.fa.Factory;
6
7 import calculadorasumas.fa.View.Entrada;
8 import calculadorasumas.fa.View.EntradaConsole;
9 import calculadorasumas.fa.View.Salida;
10 import calculadorasumas.fa.View.SalidaConsole;
11
12 /**
13  *
14  * @author
15  */
16
17 public class FabricaConsole implements FabricaEntradaSalida{
18
19     @Override
20     public Entrada crearEntrada(){
21         return (Entrada) new EntradaConsole();
22     }
23
24     @Override
25     public Salida crearSalida() {
26         return (Salida) new SalidaConsole();
27     }
28 }

```

2.1.3. Fabrica Swing:

Implementación de la interfaz abstracta FabricaEntradaSalida y crea la familia de productos enfocados en la Interfaz Gráfica de Usuario haciendo que la interacción del usuario se realice mediante componentes visuales de Java Swing.

Retorna los Productos concretos Entrada Swing y Salida Swing

```

5 package calculadorasumas.fa.Factory;
6
7 import calculadorasumas.fa.View.Entrada;
8 import calculadorasumas.fa.View.EntradaSwing;
9 import calculadorasumas.fa.View.Salida;
10 import calculadorasumas.fa.View.SalidaSwing;
11
12 /**
13  *
14  * @author
15  */
16
17 public class FabricaSwing implements FabricaEntradaSalida{
18
19     @Override
20     public Entrada crearEntrada() {
21         return (Entrada) new EntradaSwing();
22     }
23
24     @Override
25     public Salida crearSalida() {
26         return (Salida) new SalidaSwing();
27     }
28 }

```

2.2. Implementación de los Productos:

Los productos de la Calculadora son los que determinan como se reciben y devuelven lo datos, como productos Abstractos se definen dos interfaces que deben ser implementados por cada familia, **Entrada** y **Salida**:

```

package calculadorasumas.fa.View;

/**
 *
 * @author
 */
public interface Entrada {
    public String leer();
}

```

```

package calculadorasumas.fa.View;

/**
 *
 * @author
 */
public interface Salida {
    public void salidaNumero(int resultado);
    public void salidaTexto(String texto);
}

```

Para la entrada solo se tiene el método leer, en cambio para la salida se tienen salida numero y texto, cambiando el parámetro que se recibe y para facilitar la salida de texto de la interfaz de Archivo ya que si fuera solo una en común, pediría un archivo para cada mensaje de salida.

Y cada producto concreto implementa los productos abstractos dependiendo si son salida o entrada de datos:

2.2.1. Producto Concreto Consola:

Salida:

Implementa la interfaz Salida, y los métodos reciben ya sea número o texto y lo imprime en la terminal utilizando System.out.println().

```
package calculadorasumas.fa.View;

/**
 *
 * @author
 */
public class SalidaConsola implements Salida{

    @Override
    public void salidaNumero(int resultado){
        System.out.println(resultado);
    }

    @Override
    public void salidaTexto( String texto){
        System.out.println(texto);
    }

}
```

Entrada:

Implementa la interfaz Entrada, utiliza la clase Scanner instanciada con System.in para escuchar el teclado del usuario y leer() Captura y retorna toda la línea de texto ingresada por el usuario mediante el método nextLine() del Scanner.

```
package calculadorasumas.fa.View;

import java.util.Scanner;

/**
 *
 * @author
 */
public class EntradaConsola implements Entrada{

    Scanner scanner = new Scanner(System.in);

    @Override
    public String leer(){

        String entrada = scanner.nextLine();
        return entrada;
    }

}
```

2.2.2. Producto Concreto Swing:

Salida:

Implementa la interfaz Salida, y convierte la entrada del usuario y lo muestra en pantalla mediante una ventana emergente de mensaje usando JOptionPane.showMessageDialog.

```
package calculadorasumas.fa.View;

import javax.swing.JOptionPane;

/**
 *
 * @author
 */
public class SalidaSwing implements Salida{

    @Override
    public void salidaNumero(int resultado) {

        JOptionPane.showMessageDialog(null, resultado);

    }

    @Override
    public void salidaTexto(String texto) {

        JOptionPane.showMessageDialog(null, texto);

    }

}
```

Entrada:

Implementa la interfaz Entrada, utiliza la clase JOptionPane de la biblioteca Swing para generar un cuadro de diálogo de entrada (showInputDialog) y retorna el valor digitado.

```
package calculadorasumas.fa.View;

import javax.swing.JOptionPane;

/**
 *
 * @author
 */
public class EntradaSwing implements Entrada{

    @Override
    public String leer() {

        String entrada = JOptionPane.showInputDialog("Digite un numero. ");
        return entrada;

    }

}
```

2.2.3. Producto Concreto Archivo:

Salida:

Implementa la interfaz Salida, para la Salida del numero abre una ventana de guardado de explorador de archivos mediante JFileChooser, showSaveDialog y escribe el número convertido a texto dentro del archivo seleccionado utilizando Files.writeString

El método salidaTexto, a diferencia de la salida numérica, este método muestra el texto directamente en una ventana gráfica de tipo JOptionPane.showMessageDialog en lugar de

guardarlo en un archivo físico ya que de lo contrario se tendría que pedir un archivo para cada salida de texto.

```
package calculadorasumas.fa.View;

import java.io.IOException;
import java.nio.file.Files;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;

/**
 *
 * @author
 */
public class SalidaArchivo implements Salida{

    @Override
    public void salidaNumero(int resultado) {

        JFileChooser buscador = new JFileChooser();

        if (buscador.showSaveDialog(null) == JFileChooser.APPROVE_OPTION) {
            try {
                Files.writeString(buscador.getSelectedFile().toPath(), String.valueOf(resultado));
            } catch (IOException e) {
                System.out.println("Error " + e.getMessage());
            }
        }
    }

    @Override
    public void salidaTexto(String texto) {
        JOptionPane.showMessageDialog(null, texto);
    }
}
```

Entrada:

Implementa la interfaz Entrada, lo que hace el método leer es abrir un explorador de archivos utilizando JFileChooser luego el sistema lee todo el contenido del archivo usando Files.readString y lo retorna. En caso de fallar la lectura (excepción IOException) o si no se selecciona nada, retorna una cadena indicando un "Error".

```
package calculadorasumas.fa.View;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import javax.swing.JFileChooser;

/**
 *
 * @author
 */
public class EntradaArchivo implements Entrada{

    private JFileChooser buscador = new JFileChooser();

    @Override
    public String leer() {

        int resultado = buscador.showOpenDialog(null);
        if (resultado == JFileChooser.APPROVE_OPTION) {
            File archivo = buscador.getSelectedFile();
            try {
                return Files.readString(archivo.toPath());
            } catch (IOException e) {
                return "Error " + e.getMessage();
            }
        }

        return "Error";
    }
}
```

2.3. Implementación de las Operaciones:

Para la lógica de los cálculos se define un objeto abstracto por medio de una interfaz **Operación** que lo que hace es a partir de dos números calcular la operación y retornar otro número:

```
package calculadorasumas.fa.Model;

/**
 *
 * @author natha
 */
public interface Operacion {
    public int calcular(int a, int b);
}
```

Y sus dos implementaciones **Suma** y **Multiplicación**:

2.3.1. Suma :

Tiene el método Calcular que recibe dos operandos enteros y retorna el resultado aritmético de sumarlos ($a + b$).

```
5 package calculadorasumas.fa.Model;
6
7 /**
8  *
9  * @author natha
10 */
11 public class Suma implements Operacion{
12
13     @Override
14     public int calcular(int a, int b){
15         return a+b;
16     }
17 }
18 }
```

2.3.2. Multiplicación:

implementa la interfaz Operacion. e intenta resolver una multiplicación sin utilizar el operador aritmético, sino empleando sumas, instanciando un objeto de la clase suma.

```
5 package calculadorasumas.fa.Model;
6
7 /**
8  *
9  * @author natha
10 */
11 public class Multiplicacion implements Operacion{
12
13     @Override
14     public int calcular (int a, int b){
15         int multiplicacion=0;
16         int bPositivo=Math.abs(b);
17
18         Operacion objSuma =new Suma();
19
20         for (int i=0; i<bPositivo; i++){
21             multiplicacion= objSuma.calcular(multiplicacion, a);
22         }
23         if(b>=0){
24             return multiplicacion;
25         }else{
26             return multiplicacion*(-1);
27         }
28     }
29 }
30 }
```

3. Código Cliente Explicación:

La clase **Cliente** funciona como el orquestador principal que delega las tareas a los productos abstractos, recibe cualquier fabrica y cualquier operación:

```
package calculadorasumas.fa;

import calculadorasumas.fa.Factory.FabricaEntradaSalida;
import calculadorasumas.fa.Model.Operacion;
import calculadorasumas.fa.View.Entrada;
import calculadorasumas.fa.View.Salida;

/**
 *
 * @author
 */
public class Cliente {

    public Entrada objEntrada;
    public Salida objSalida;
    public Operacion objOperacion;

    public Cliente(FabricaEntradaSalida fabrica, Operacion operacion) {
        this.objEntrada = fabrica.crearEntrada();
        this.objSalida = fabrica.crearSalida();
        this.objOperacion = operacion;
    }

    public String leer() {
        return objEntrada.leer();
    }

    public void enviarTexto(String texto) {
        objSalida.salidaTexto(texto);
    }

    public void enviarNumero(int resultado) {
        objSalida.salidaNumero(resultado);
    }

    public int calcular (int a, int b) {
        return objOperacion.calcular(a, b);
    }
}
```

El **Main** actúa como el punto de inicio de la aplicación, determinando qué familia de productos se va a construir dependiendo de la respuesta del usuario:

```
public class Main {

    public static void main(String[] args) {

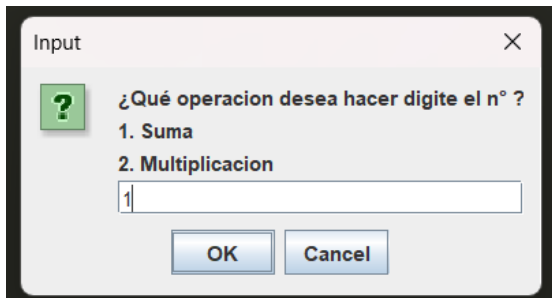
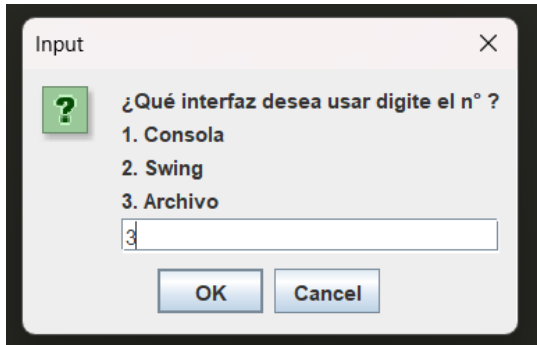
        String opcion = JOptionPane.showInputDialog(
            "¿Qué interfaz desea usar digite el n° ?\n1. Consola\n2. Swing\n3. Archivo"
        );
        String calculo = JOptionPane.showInputDialog(
            "¿Qué operación desea hacer digite el n° ?\n1. Suma\n2. Multiplicación"
        );

        FabricaEntradaSalida fabrica;
        Operacion objOperacion;
        if ("1".equals(opcion)){
            fabrica = new FabricaConsola();
        }
        else if ("2".equals(opcion)) {
            fabrica = new FabricaSwing();
        }
        else{
            fabrica = new FabricaArchivo();
        }

        if ("1".equals(calculo)){
            objOperacion = new Suma();
        }
        else {
            objOperacion = new Multiplicacion();
        }

        Cliente cliente = new Cliente(fabrica, objOperacion);
        cliente.enviarTexto("----- Calculadora -----");
        cliente.enviarTexto(" Digite un numero");
        int a = Integer.parseInt(cliente.leer());
        cliente.enviarTexto(" Digite otro numero");
        int b = Integer.parseInt(cliente.leer());
        int resultado= cliente.calcular(a, b);
        cliente.enviarTexto(" Resultado:");
        cliente.enviarNumero(resultado);
    }
}
```

Con un Show Input Dialog de swing se recibe la opción del usuario y dependiendo de eso se crea la fabrica concreta y la operación que se quiera:



4. Pruebas para cada producto y operación:

Se hicieron las respectivas pruebas para cada producto.

```
package calculadorasumas.fa.Pruebas;

import calculadorasumas.fa.Cliente;
import calculadorasumas.fa.Factory.FabricaArchivo;
import calculadorasumas.fa.Factory.FabricaEntradaSalida;
import calculadorasumas.fa.Model.Operacion;
import calculadorasumas.fa.Model.Suma;

/**
 *
 * @author natha
 */
public class PruebaClienteArchivoSuma {

    public static void main(String[] args){
        FabricaEntradaSalida fabrica = new FabricaArchivo();
        Operacion suma = new Suma();
        Cliente cliente = new Cliente(fabrica,suma);

        cliente.enviarTexto(" ---- Calculadora ----");
        cliente.enviarTexto(" Digite un numero");
        int a = Integer.parseInt(cliente.leer());
        cliente.enviarTexto(" Digite otro numero");
        int b = Integer.parseInt(cliente.leer());
        int resultado= cliente.calcular(a, b);
        cliente.enviarTexto(" Resultado:");
        cliente.enviarNumero(resultado);
    }
}
```

```
package calculadorasumas.fa.Pruebas;

import calculadorasumas.fa.Cliente;
import calculadorasumas.fa.Factory.FabricaArchivo;
import calculadorasumas.fa.Factory.FabricaEntradaSalida;
import calculadorasumas.fa.Model.Multiplicacion;
import calculadorasumas.fa.Model.Operacion;

/**
 *
 * @author natha
 */
public class PruebaClienteArchivoMult {

    public static void main(String[] args){
        FabricaEntradaSalida fabrica = new FabricaArchivo();
        Operacion multiplicacion = new Multiplicacion();
        Cliente cliente = new Cliente(fabrica,multiplicacion);

        cliente.enviarTexto(" ---- Calculadora ----");
        cliente.enviarTexto(" Digite un numero");
        int a = Integer.parseInt(cliente.leer());
        cliente.enviarTexto(" Digite otro numero");
        int b = Integer.parseInt(cliente.leer());
        int resultado= cliente.calcular(a, b);
        cliente.enviarTexto(" Resultado:");
        cliente.enviarNumero(resultado);
    }
}
```

4.1.Consola:

Salidas paso a paso de Suma:

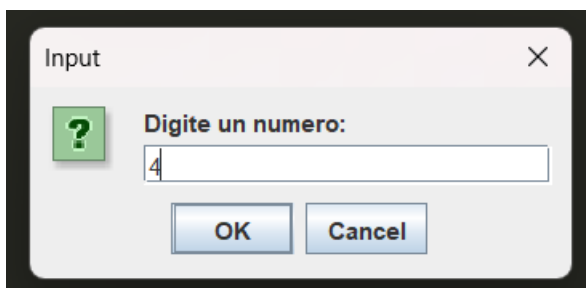
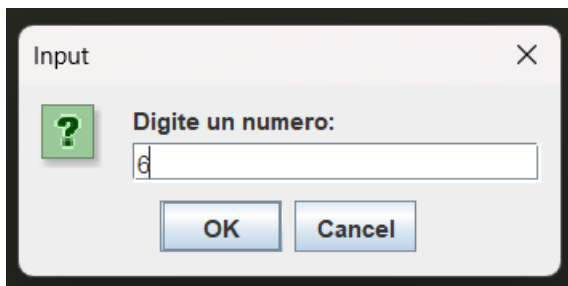
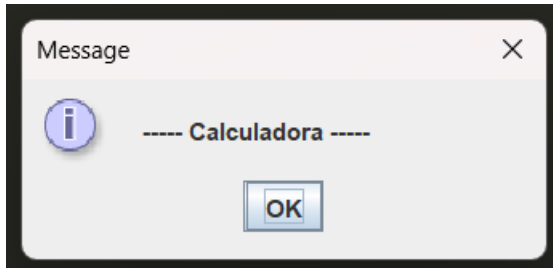
```
run:
  ---- Calculadora ----
  Digite un numero
9
  Digite otro numero
1
  Resultado:
10
BUILD SUCCESSFUL (total time: 52 seconds)
|
```

multiplicación:

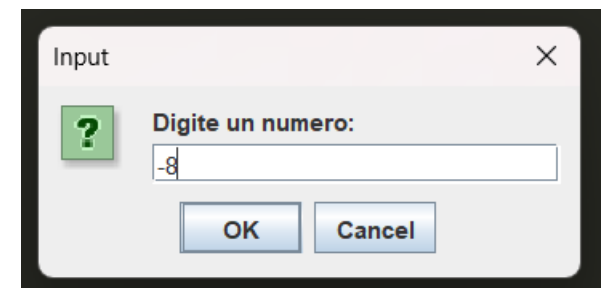
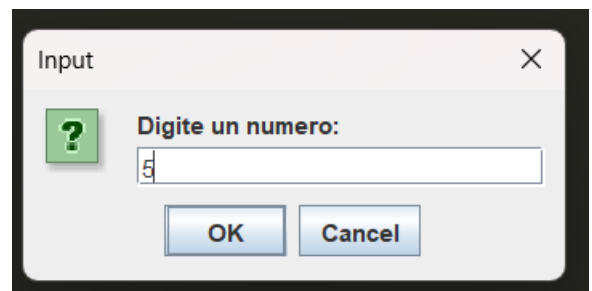
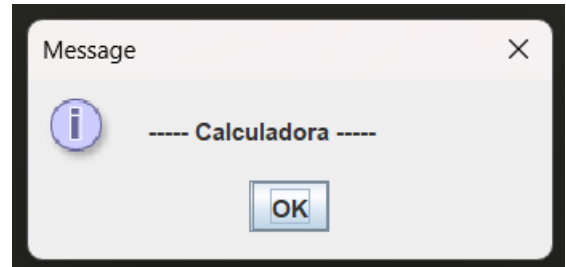
```
---- Calculadora ----
Digite un numero
8
  Digite otro numero
5
  Resultado:
40
BUILD SUCCESSFUL (total time: 39 seconds)
```

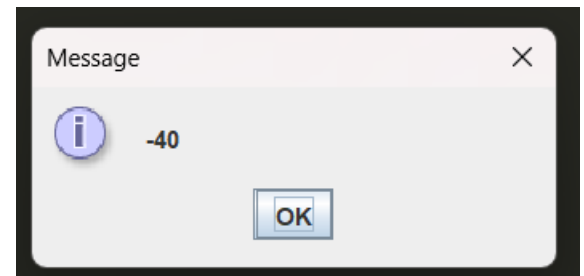
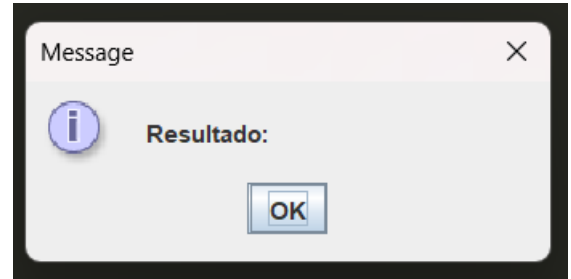
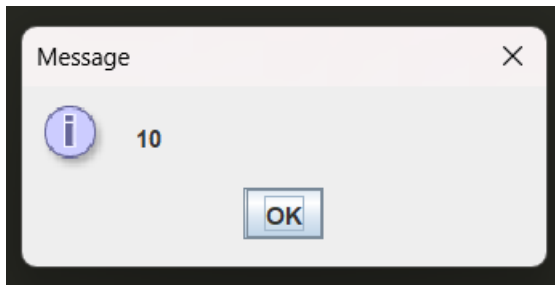
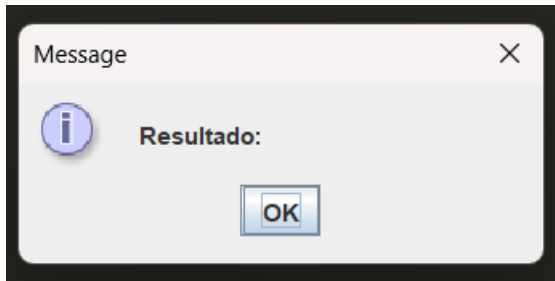
4.2.Swing:

Salidas paso a paso de Suma:



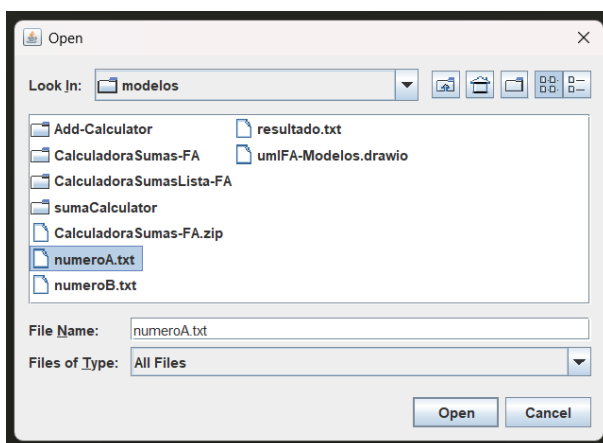
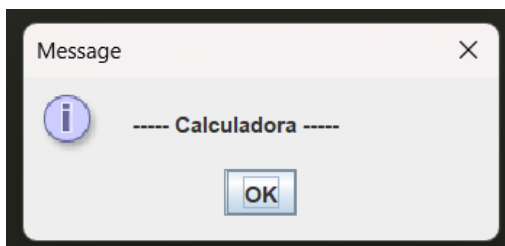
multiplicación:



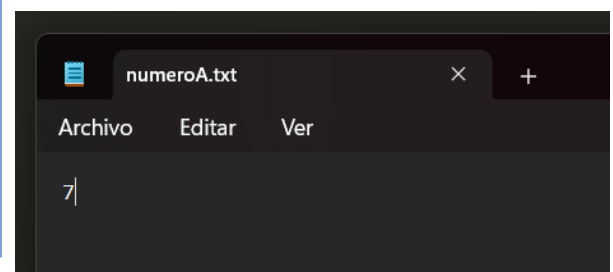


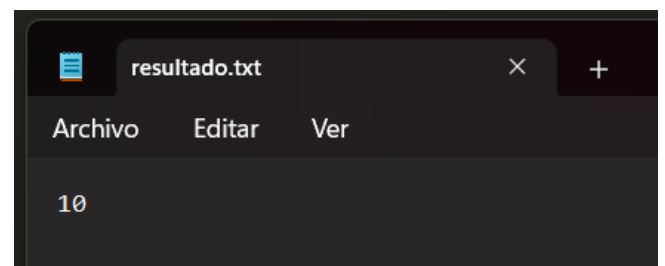
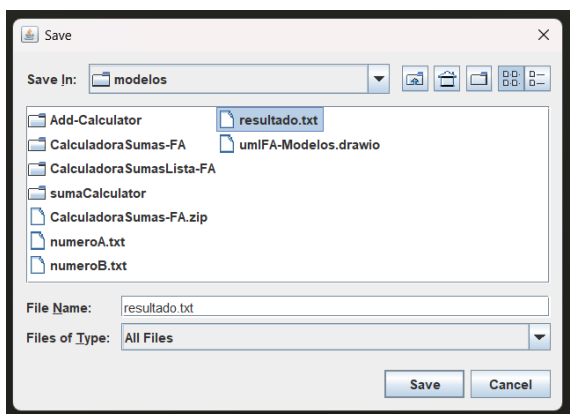
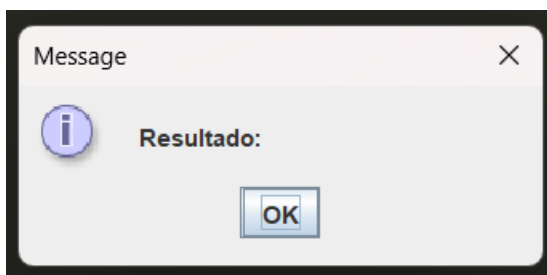
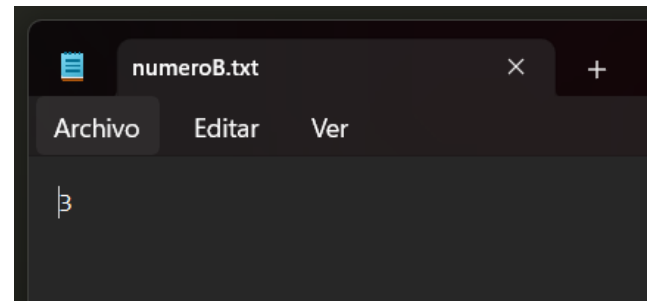
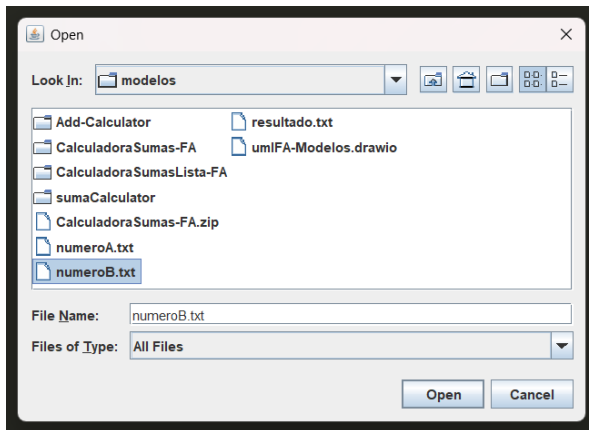
4.3.Archivo:

Salidas paso a paso de Suma:



Contenido del txt:





5. Código:

5.1.Link del repositorio:

<https://github.com/Thalyhx/AbstractFactoryCalculator.git>

5.2.Estructura de Paquetes:

